```
$car1 = [
    'make' => 'VW',
    'model' => 'Golf',
    'variant' => 'GTI',
    'colour' => 'Oak Green',
];

$car2 = [
    'X_MAKE' => 'VW',
    'X_MODEL' => 'Golf',
    'X_VARIANT' => 'GTI 16v',
    'X_COLOUR' => 'Oak Green',
];

print_r(array_diff_uassoc($car1, $car2, 'keyComparison'));

function keyComparison ($k1, $k2) {
    return strcasecmp($k1, ltrim($k2, 'X_'));
}

Output:
Array
(
    [variant] => GTI
)
```

In addition to a mismatch of case, we've also prepended each key in `$car2` with `'X_'`: simply changing the case of the keys won't be enough here. By using our own key comparison function we can first clean up the keys of `$car2` by stripping the leading `'X_'` characters with PHP's built-in `ltrim()` function. We also make use of `strcasecomp()`, another built-in which allows case-insensitive string comparison, and returns `1`-, `0`, or `1` which is exactly what our callback needs to do as well. (For more details on those two string functions see Chapter 8, "Strings.") Note that the two arguments to `keyComparison()` are the keys (strings) of `$car1` and `$car2`. Value comparison callbacks are given the elements being compared; key comparison callbacks are given the keys. One more point to make here: our callback is now a named function, so we pass its name as a string as the `callable` argument.